华东师范大学软件学院实验报告

实验课程：计算机网络　　年级：2024 级　　实验成绩：
实验名称: ARP　　姓名：刘小凡　　实验日期：2026-11-03
实验编号：Lab4　　学号：10244602411　　实验时间：2 学时

一、实验目的

本次实验围绕 ARP（Address Resolution Protocol）展开，目的是通过抓取并逐字段检查真实的 ARP 报文，理解 IPv4 地址如何解析为本链路的以太网地址，掌握 ARP 报文与承载它的以太网帧之间的关系。具体目标如下：

1. 掌握用 ipconfig / 路由表获取本机以太网地址与默认网关的方法，并据此确定捕获接口和过滤条件；

2. 掌握以 arp 为捕获过滤器、关闭混杂模式捕获 ARP 流量的完整流程，能够保存可复核的 pcapng 迹线；

3. 能区分 ARP 请求与应答，逐字段读取 28 字节 ARP 报文的九个字段，并说明请求与应答在这四个地址字段上的对应关系；

4. 能把 ARP 报文与以太网帧分开：以太网 14 B 首部、28 B ARP 报文、以太网填充，并解释 42 B 与 60 B 两种帧长的来源；

5. 能回答手册 Step 4 的五道问答题，并画出本机与默认网关之间请求/应答的交换示意图；

6. 能识别迹线中 ARP 探测（Probe）与免费 ARP 通告（Announcement）等常见行为，并说明它们的用途。

二、实验内容与实验步骤

实验按手册的四个 Step 推进：先核对接口地址与默认网关并设置捕获过滤器，再捕获 ARP 流量并定位一次完整的请求/应答，然后画出交换示意图，最后依据报文回答 Step 4 的五道问题。

Step 1 取址与捕获：用 ipconfig /all 读出 WLAN 接口的 IPv4 地址与物理地址，用路由表确认默认网关，随后在 Wireshark 中选择该接口、捕获过滤器填 arp、关闭混杂模式并开始捕获。手册建议先清空 ARP 缓存再访问网页以触发解析，本次的实际做法与偏差在第 4.1 节如实说明。

Step 2 定位迹线：按手册要求以本机以太网地址设置显示过滤器（形如 eth.addr == 14:ac:60:56:df:85），把视图限制在与本机相关的 ARP 报文，再分别展开一条请求和一条应答，读取全部字段。

Step 3 绘制交换示意图：利用 Step 2 读到的四个地址字段，画出本机与默认网关之间的请求与应答，标注每个报文的发送方与目标 MAC/IP，并圈出本机原本不知道、由应答学到的那个以太网地址。

Step 4 报文细节：针对以太网封装与 ARP 报文字段，回答操作码取值、ARP 报文长度、未知目标 MAC 的填充值、以太网类型值以及应答是否广播这五道问题。

本次实测的迹线 Lab4_ARP.pcapng 共 53 帧，全部为 ARP，其中请求 40 帧、应答 13 帧。

三、实验环境

项目	实测配置
操作系统	Windows 11 25H2（build 26200），64 位
抓包与分析	Wireshark / TShark 4.6.8，Npcap 1.88

项目	实测配置
命令行工具	ipconfig、路由表查询、arp、ping
捕获接口	WLAN：MediaTek Wi-Fi 6E MT7922 (RZ616) 160MHz Wireless LAN Card
本机地址	IPv4 172.31.240.50/16；MAC 14:ac:60:56:df:85
默认网关	172.31.0.1，其 MAC 54:c6:ff:7b:38:02
捕获过滤器	arp（关闭混杂模式）
迹线文件	Lab4_ARP.pcapng，53 帧

四、实验过程与分析

本节只记录实验过程与实测现象：每一步做了什么、看到什么结果、遇到什么偏差以及如何解释。手册要求提交的题目与答案统一放在第五节，便于对照评分。

4.1 网络环境与捕获方法

先用 ipconfig /all 读出 WLAN 接口的状态，如图 1：本机 IPv4 地址为 172.31.240.50（前缀长度 16），物理地址为 14:ac:60:56:df:85，默认网关为 172.31.0.1。这三个值是后续设置过滤器和判读报文的基准。

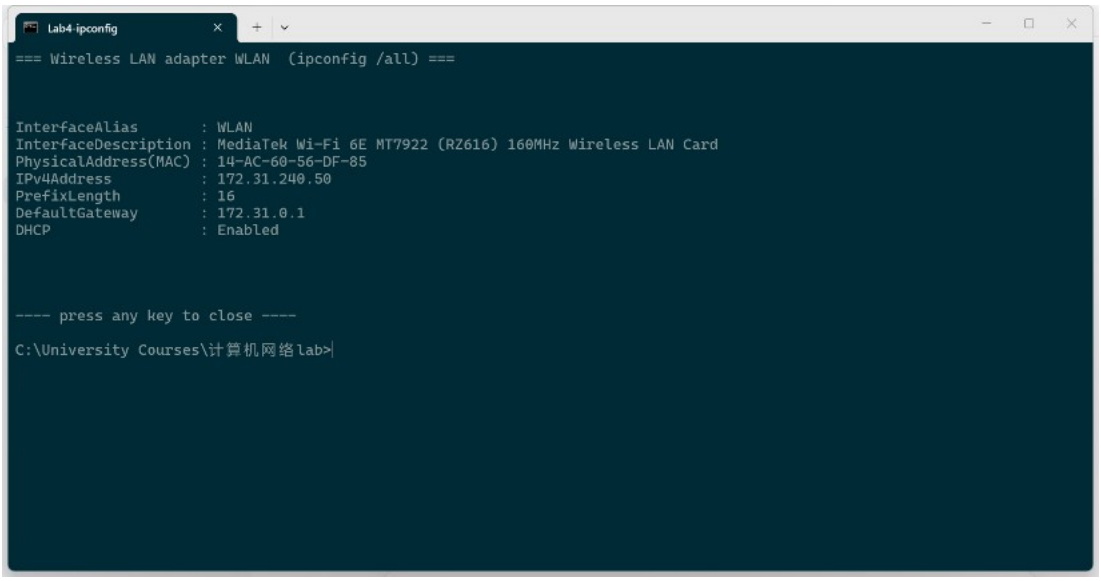


图 1 WLAN 接口的 IPv4 地址、物理地址与默认网关（对应手册 Step 1 的 ipconfig 取址）

再用路由表确认默认路由确实指向该网关，如图 2。需要指出的是，本机同时存在 singbox_tun 的默认路由（下一跳 172.18.0.2），因此不能假定系统所有流量都经 WLAN 转发。但 ARP 只用于解析同一广播域内的地址，隧道不会替本机解析 WLAN 网段的网关，所以本实验把捕获接口明确指定为 WLAN，结果不受隧道影响。

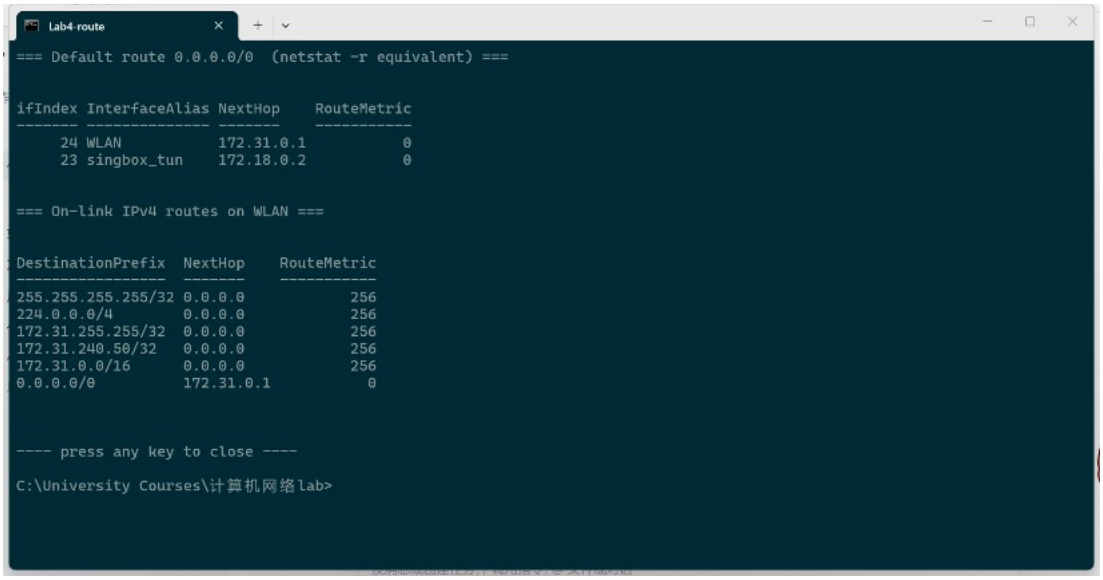


图 2 默认路由 0.0.0.0/0：WLAN 经 172.31.0.1，另有 singbox_tun 的并行默认路由

查看 ARP 缓存可以确认网关的解析结果，如图 3：接口 172.31.240.50 下存在一条 172.31.0.1 → 54:c6:ff:7b:38:02 的动态（dynamic）条目，其余多为组播/广播的静态条目。这条动态条目正是 ARP 协议工作后留下的产物，可与迹线中的应答互相印证。

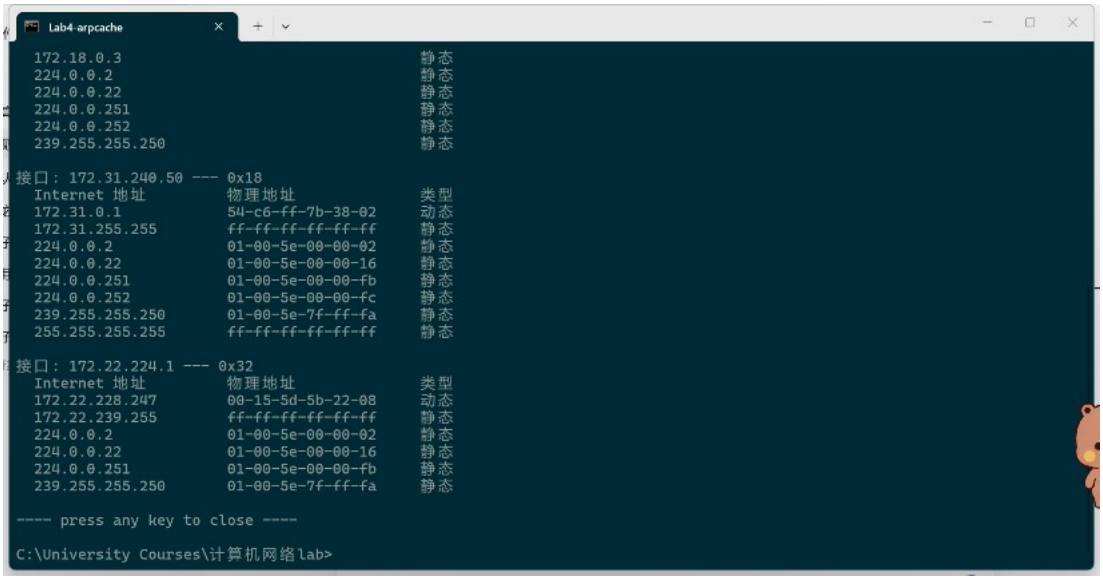


图 3 arp -a 输出：WLAN 接口下网关 172.31.0.1 对应 54-c6-ff-7b-38-02（dynamic）

关于手册建议的“先清空 ARP 缓存再访问网页”，需要如实说明本次的偏差：清空缓存（arp -d）需要管理员权限，本次以普通用户身份运行，命令无法执行，因此没有清空缓存；工作负载也不是浏览器访问网页，而是绑定了 WLAN 源地址的 ping（ping -S 172.31.240.50 172.31.0.1）。捕获期间迹线中自然出现了对网关的请求与应答，足以完成后续全部字段与交换分析；但由于没有清空缓存，不能断言第 1 帧的请求一定由那次 ping 触发，也不能据此推断解析的触发时机。

```
# 绑定 WLAN 源地址访问网关（本次使用的工作负载）
ping -S 172.31.240.50 172.31.0.1


# 查看 ARP 缓存（无需管理员权限）
arp -a
```

4.2 ARP 迹线的定位

捕获得到的 Lab4_ARP.pcapng 共 53 帧，协议分层显示 100% 为 ARP，说明捕获过滤器生效、没有混入其他协议。图 4 为迹线概览，可以看到 Info 列同时出现 “Who has …? Tell …”（请求）与 “… is at …”（应答），以及少数标注为 ARP Probe 与 ARP Announcement 的报文。整条迹线共 40 个请求、13 个应答。

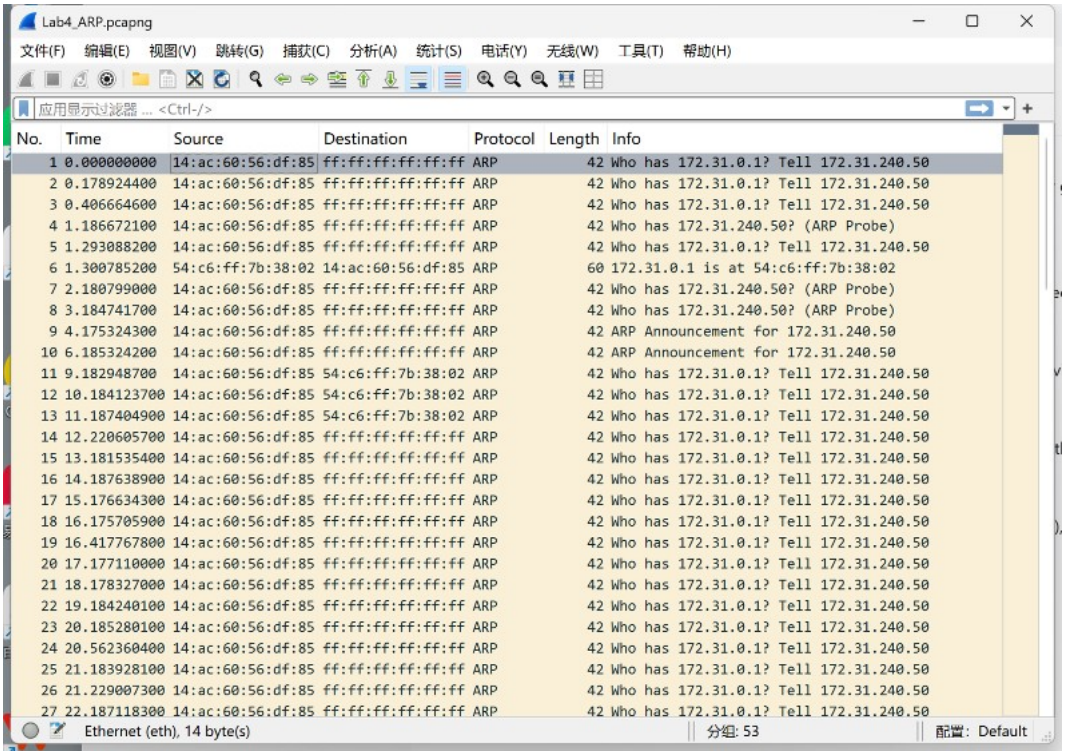


图 4 迹线概览：全部 53 帧均为 ARP，含请求、应答、探测与通告（来源：Lab4_ARP.pcapng，状态栏显示合计 53 帧）

按手册 Step 2 的要求，以本机以太网地址设置显示过滤器 eth.addr == 14:ac:60:56:df:85，结果如图 5。这里出现了一个与手册预期不同的现象：过滤后仍是 53 帧，即 100% 的报文都与本机相关，并没有像手册示例那样筛掉一大半。原因是本迹线在本机接口上捕获且全部为 ARP，而每个 ARP 报文的发送方或目标以太网地址中都有本机地址（本机发出的请求、发给本机的应答、以及本机自身的探测与通告）。因此这条过滤器在本迹线中起“确认”而非“筛选”作用；若要缩小视图，应按 IP 或操作码过滤。

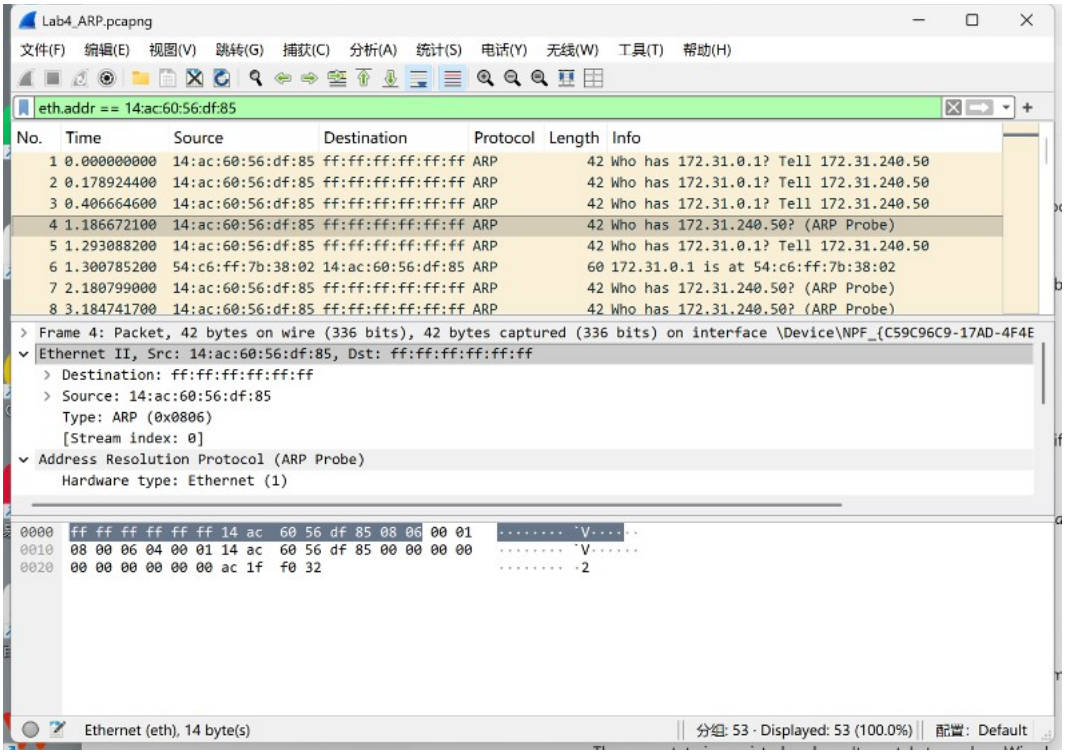


图 5　按本机以太网地址过滤 eth.addr == 14:ac:60:56:df:85：本迹线 53 帧全部与本机相关（状态栏 Displayed 53/100.0%）

类别	数量	帧号示例	判据
请求（request）	40	1、2、3、5、11…	opcode = 1，Info 为 Who has …
应答（reply）	13	6、14、51、53…	opcode = 2，Info 为 … is at …
ARP 探测（Probe）	6	4、7、8、33、35、46	发送方 IP 为 0.0.0.0，用于地址冲突检测
ARP 通告（Announcement）	4	9、10、48、49	发送方 IP 与目标 IP 相同（免费 ARP）

表 1　迹线中各类 ARP 报文的构成（分类由 analyze.py 依据原始字节判定）

4.3　以太网封装与 ARP 报文字节结构

ARP 报文并不直接出现在链路上，而是作为以太网帧的载荷。图 6 把以太网首部展开：目的地址 ff:ff:ff:ff:ff:ff、源地址 14:ac:60:56:df:85、类型 ARP (0x0806)，右侧字节窗口中被高亮的正是 08 06 两个字节；状态栏同时给出“Ethernet (eth), 14 byte(s)”，即以太网首部 14 字节。

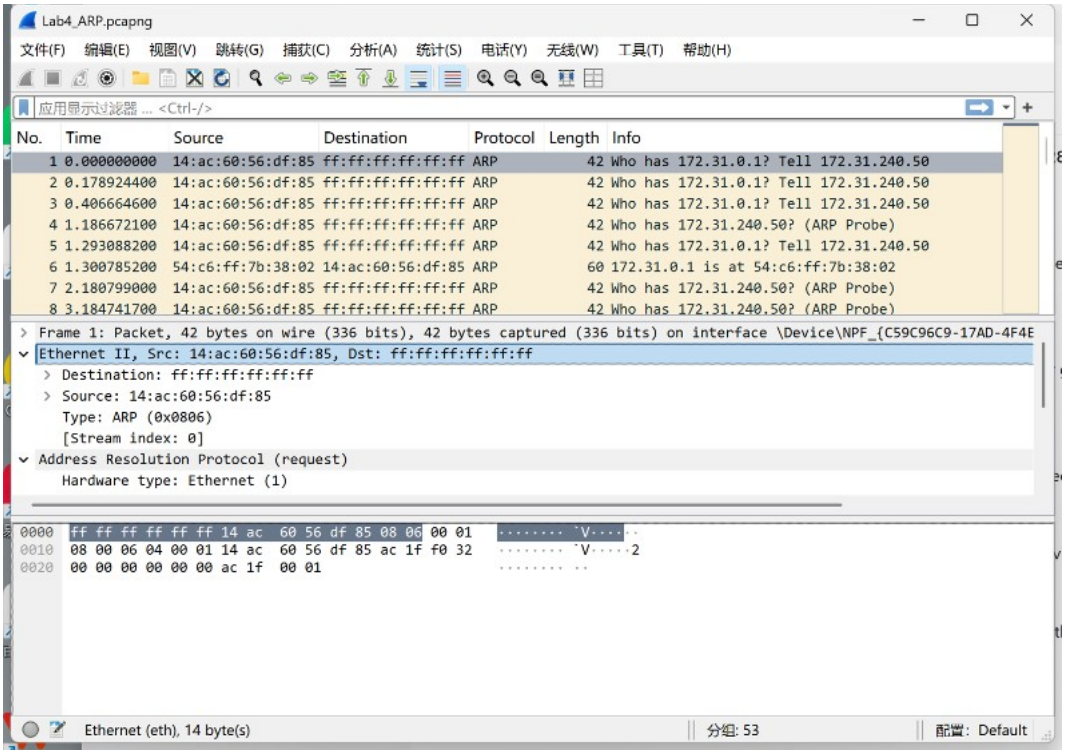


图 6　Ethernet II 首部展开：Destination 为广播、Type 为 ARP (0x0806)，字节窗口高亮 08 06（来源：Lab4_ARP.pcapng 第 1 帧）

把以太网首部与 ARP 报文分开之后，两条报文的长度就能对上账：ARP 报文本身固定为 28 字节（两个 2 字节类型 + 两个 1 字节长度 + 一个 2 字节操作码 + 四段地址），以太网首部 14 字节；最小以太网帧为 60 字节（不含前导码与 FCS），因此 42 字节的请求不需要填充，而应答补齐到 60 字节，多出的 18 字节是以太网填充而不是 ARP 字段。

报文	帧长	以太网首部	ARP 报文	以太网填充	核对
请求（第 1 帧）	42 B	14 B	28 B	0 B	14 + 28 = 42
应答（第 6 帧）	60 B	14 B	28 B	18 B	14 + 28 + 18 = 60

表 2　帧长的分解：ARP 报文均为 28 B，差异来自以太网填充

本机发出的请求（第 1 帧）的完整字段见图 7。可以看到 Hardware type 为 Ethernet (1)、Protocol type 为 IPv4 (0x0800)、Hardware size 6、Protocol size 4、Opcode 为 request (1)；四个地址字段中，发送方 MAC/IP 已填本机值，目标 IP 填网关地址，而目标 MAC 填全零。状态栏显示该 ARP 报文为 28 byte(s)。

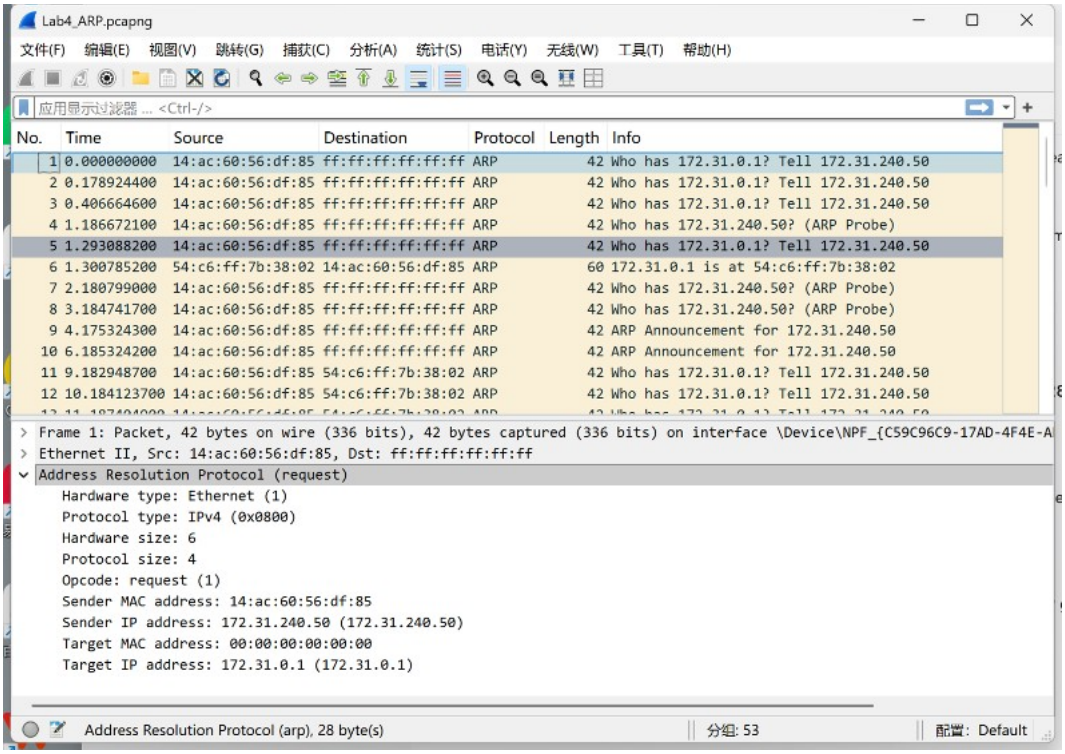


图 7　ARP 请求（第 1 帧）的全部字段：Opcode = request (1)，Target MAC = 00:00:00:00:00:00，状态栏显示 ARP 报文 28 字节

网关回的应答（第 6 帧）见图 8。Hardware/Protocol 类型与长度保持不变，只有 Opcode 变为 reply (2)；四个地址字段整体“换位”：原来的目标（网关）成为发送方，原来的发送方（本机）成为目标，并且此时两对地址都已填满。该帧的以太网目的地址是本机 MAC，属单播。

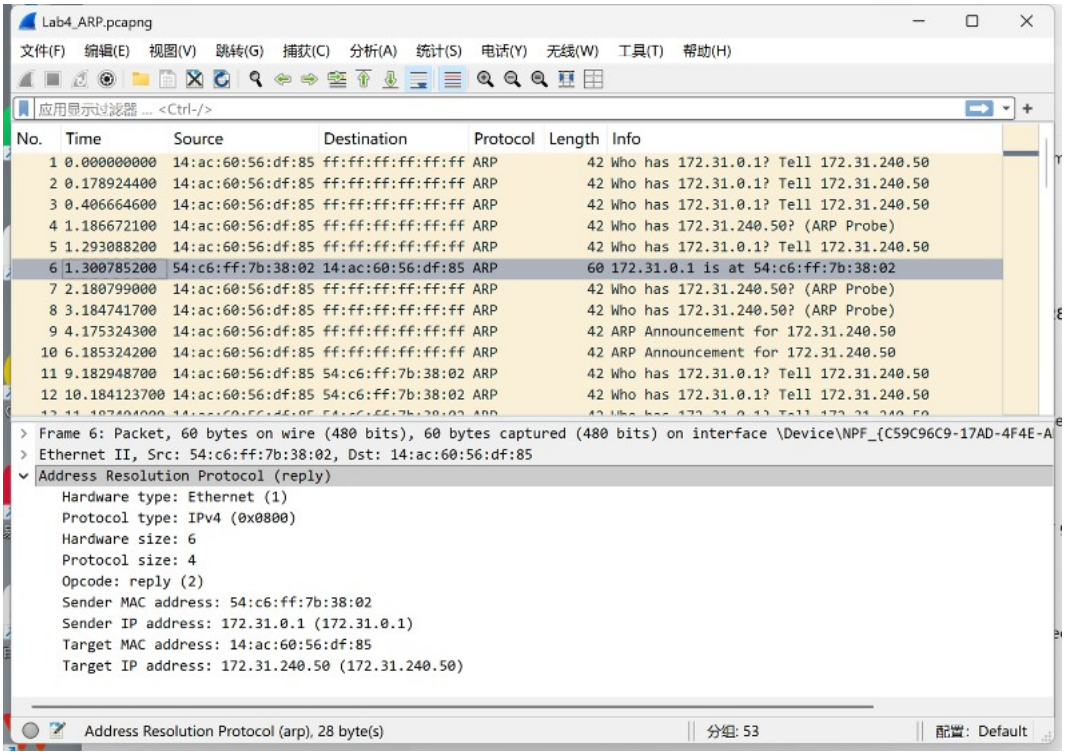


图 8　ARP 应答（第 6 帧）的全部字段：Opcode = reply (2)，四个地址均已填满，以太网目的为本机单播地址

这里最容易混淆的是两个层次的“地址”：以太网首部的目的地址决定这一帧在链路上被谁收下，而 ARP 报文内部的目标 MAC 是“想知道的那个地址”。请求把前者设为广播、后者设为零，正是为了让全网都能收到、同时表明自己还不知道答案。

4.4 迹线中的其他 ARP 行为

除了一问一答，迹线里还有两类值得记录的行为。用过滤器 arp.isprobe || arp.isannouncement 可把它们筛出来，共 10 帧（图 9）：6 个 ARP 探测（帧 4、7、8、33、35、46）与 4 个 ARP 通告（帧 9、10、48、49）。探测报文的发送方 IP 为 0.0.0.0、目标是本机自己的地址，作用是地址冲突检测；通告报文的发送方 IP 与目标 IP 相同（都是 172.31.240.50），属于免费 ARP，用于在地址启用后通知同网段其他主机更新缓存。二者都是 RFC 5227 定义的行为，不是异常。

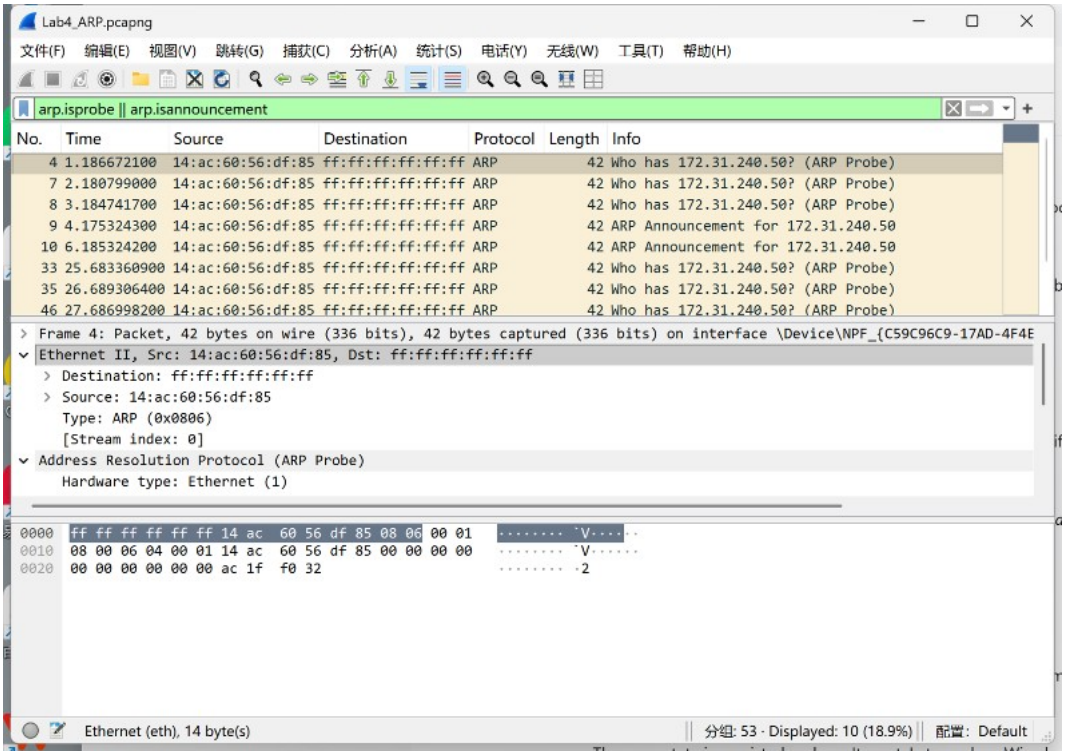


图 9 ARP 探测与免费 ARP 通告：过滤器 arp.isprobe || arp.isannouncement 筛出 10 帧（状态栏 Displayed 10/18.9%）

另一个值得记录的异常是第 11、12、13 帧：它们同样是询问 172.31.0.1 的请求，但以太网目的地址不是广播，而是网关的单播 MAC 54:c6:ff:7b:38:02。这与“ARP 请求总是广播”的常见说法不符，说明该说法应表述为“正常情况下广播”：在已经知道对方硬件地址、或实现选择直接单播以减轻广播负载时，也会出现单播的请求。因此判断一个请求是否广播，要看具体帧的以太网目的地址，而不能只凭报文类型下结论。

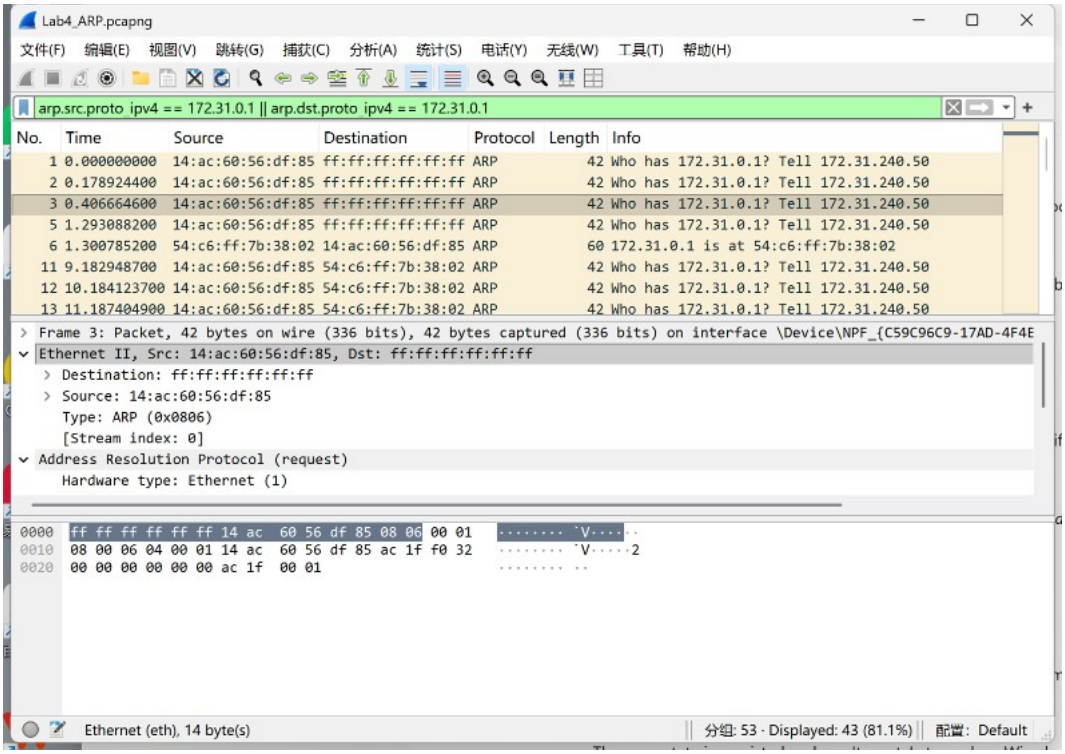


图 10　与网关相关的全部 ARP 报文（过滤器按 172.31.0.1 匹配，共 43 帧），其中第 11–13 帧为以太网目的为单播的请求

五、必做题目与解答

本节集中列出手册要求提交的两项 Turn-in 与本次作答结果。题目原文以「题目原文」标出，答案紧接其后。

5.1　必做题目一：绘制 ARP 交换示意图（Turn-in ①）

题目原文（Step 3）：画出本机与默认网关之间的 ARP 请求与应答，标注一个报文为请求、另一个为应答；给出每个报文的发送方与目标 MAC 与 IP 地址；最后在图上圈出本机所求的那个以太网地址，说明它在交换中从哪里得到。

依据第 1 帧与第 6 帧实测字段绘制的交换示意图如图 11。图中上方为请求：以太网目的为广播 ff:ff:ff:ff:ff:ff，ARP 内发送方是本机 14:ac:60:56:df:85 / 172.31.240.50，目标是 172.31.0.1，目标 MAC 填零；下方为应答：以太网目的改为本机单播，ARP 内发送方变为网关 54:c6:ff:7b:38:02 / 172.31.0.1，目标变为本机。



图 11　本机与默认网关之间的 ARP 请求与应答（依据 Lab4_ARP.pcapng 第 1、6 帧实测绘制；红圈为本机所求、由应答学到的网关 MAC）

所求的以太网地址是 54:c6:ff:7b:38:02。它来自应答报文的“发送方 MAC”字段：网关在回送应答时把自己的硬件地址填进该字段，本机收到后即可把它与请求里的目标 IP 172.31.0.1 绑定，写进 ARP 缓存（图 3 的动态条目即此结果）。示意图中把这一格用红圈标出，对应请求中取值为全零的“目标 MAC”，二者是同一个待求量的“未知”与“已知”两种状态。

5.2　必做题目二：ARP 报文细节五问（Turn-in ②）

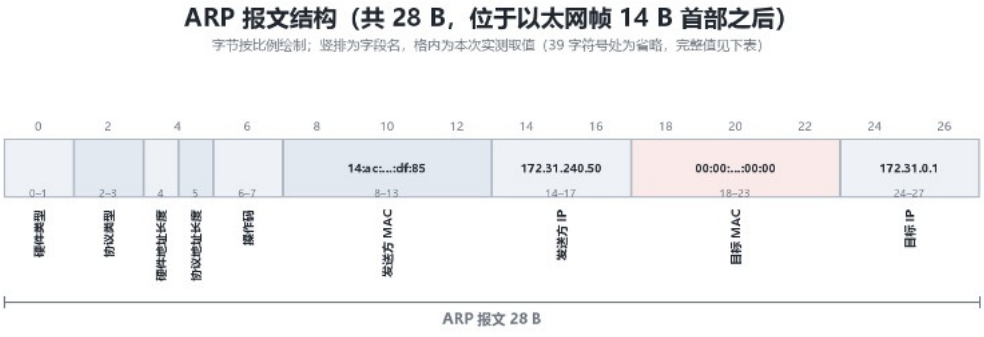
题目原文（Step 4）：检查一条 ARP 请求与一条 ARP 应答，回答五个问题——请求与应答各用什么操作码？ARP 首部有多大？请求中未知的目标 MAC 填什么值？哪个以太网类型值表示上层协议是 ARP？ARP 应答是广播还是不像请求那样广播？

为便于对照，先把五问的答案汇总在表 3，随后逐题给出依据。ARP 报文的字节布局与两帧的实测取值见图 12。

问题	答案	依据
1. 请求/应答的操作码	请求 = 1，应答 = 2	第 1 帧 Opcode: request (1)；第 6 帧 Opcode: reply (2)
2. ARP 首部大小	两者均为 28 字节	两帧状态栏均为 28 byte(s)；8 B 固定字段 + 4 段地址
3. 未知目标 MAC 的取值	全零 00:00:00:00:00:00	第 1 帧 Target MAC address 字段
4. 表示 ARP 的以太网类型	0x0806（Ethernet II 的 Type 字段）	图 6 中 Type: ARP (0x0806)，字节窗口 08 06
5. 应答是否广播	不是广播，是单播到请求方	第 6 帧以太网目的为本机 MAC 14:ac:60:56:df:85

表 3　Step 4 五问的答案与依据

ARP 报文结构的示意图见图 12：它把 28 个字节按比例展开，并列出一条请求与一条应答在每个字段上的实际取值，逐题作答时可与它逐行对照。



字节偏移	长度	字段	实测取值（请求 / 应答）
0–1	2	硬件类型	0x0001 / 0x0001
2–3	2	协议类型	0x0800 / 0x0800
4	1	硬件地址长度	6 / 6
5	1	协议地址长度	4 / 4
6–7	2	操作码	0x0001（请求）/ 0x0002（应答）
8–13	6	发送方 MAC	14:ac:60:56:df:85 / 54:c6:ff:7b:38:02
14–17	4	发送方 IP	172.31.240.50 / 172.31.0.1
18–23	6	目标 MAC	00:00:00:00:00:00 / 14:ac:60:56:df:85
24–27	4	目标 IP	172.31.0.1 / 172.31.240.50

图 12　ARP 报文结构（28 B）：九个字段的字节偏移、长度与第 1、6 帧的实测取值

问题 1：请求与应答分别用什么操作码？请求的操作码为 1，应答的操作码为 2，位于 ARP 报文的第 6–7 字节。图 7 与图 8 的字段树分别显示 Opcode: request (1) 与 Opcode: reply (2)，图 12 的字节表给出的十六进制值为 0x0001 与 0x0002。由于这两个字段是大端序的 16 位整数，抓包中的字节顺序为 00 01 / 00 02。

问题 2：ARP 首部有多大？请求与应答都是 28 字节。ARP 报文的长度并不写死，而是由硬件地址长度与协议地址长度算出：8 字节固定字段（硬件类型 2 + 协议类型 2 + 硬件地址长度 1 + 协议地址长度 1 + 操作码 2），加上两段硬件地址（2 × 6 = 12）和两段协议地址（2 × 4 = 8），合计 8 + 12 + 8 = 28 字节。图 7 与图 8 的状态栏都显示“Address Resolution Protocol (arp), 28 byte(s)”。需要说明的是，28 字节是本例（以太网 + IPv4）的结果，并非所有硬件与协议的通用定值：若地址长度字段变化，ARP 报文长度也随之变化。

问题 3：请求中未知的目标 MAC 填什么值？填全零，即 00:00:00:00:00:00。ARP 没有单独的“未知”标记，只能用一个不可能作为真实单播地址的取值占位，全零正好满足这个要求。图 7 的 Target MAC address 字段显示 00:00:00:00:00:00，图 12 的字节表在偏移 18–23 处也是六个 00。特别要区分：以太网首部的目的地址同时是广播 ff:ff:ff:ff:ff:ff，那是“这一帧发给谁”，而 ARP 内的目标 MAC 是“想知道谁的地址”，两者层次不同、取值也不同。

问题 4：哪个以太网类型值表示上层是 ARP？0x0806。它占据以太网帧的第 12–13 字节，紧跟在目的地址与源地址之后。图 6 中 Ethernet II 展开后显示 Type: ARP (0x0806)，字节窗口把 08 06 两个字节高亮；图 12 的字节表也把这两个字节列在以太网首部末尾。作为对照，ARP 报文内部的 Protocol type 字段取值为 0x0800，它表示“要解析的是 IPv4 地址”，与 0x0806 是完全不同的两个字段，不能混为一谈。

问题 5：ARP 应答是广播吗？不是。请求以广播发出（以太网目的 ff:ff:ff:ff:ff:ff），而应答是单播，直接发给提出请求的主机：第 6 帧的以太网目的地址是本机 MAC 14:ac:60:56:df:85，源地址是网关 MAC 54:c6:ff:7b:38:02。这样设计的理由是请求方身份已经明确——请求报文里就带着发送方的 MAC 与 IP，网关无需再广播即可精确回复，同时也避免打扰同网段其他主机。本迹线中的 13 个应答目的地址均为单播地址，与该结论一致。

六、实验结果总结

6.1 实验结论

关于交换过程：本机在不知道网关 54:c6:ff:7b:38:02 的情况下广播请求，网关以单播应答，本机从应答的发送方 MAC 字段学到映射并写入 ARP 缓存。这条链路上的证据是第 1 帧（请求）、第 6 帧（应答）与 arp -a 中的动态条目，三者互相印证。

关于报文结构：ARP 报文为 28 字节，以太网首部为 14 字节。请求的以太网帧长 42 字节，应答被以太网填充到 60 字节，多出的 18 字节不属于 ARP 字段。把帧长直接当成 ARP 报文长度是常见错误，必须先减去以太网首部与填充。

关于字段取值：操作码请求为 1、应答为 2；以太网类型为 0x0806；请求中未知的目标 MAC 填全零；应答为单播。这四点与手册问题一一对应，且都能在原始字节中定位到具体偏移。

关于迹线的其他行为：迹线中还有 6 个地址探测与 4 个免费 ARP 通告，以及 3 个以太网目的为单播的请求。后者的存在提醒我们，“请求总是广播”只是常态而非定律，结论应保留适用条件。

6.2 实验体会

这次实验最主要的收获是分清了两组容易混淆的地址：以太网首部的目的/源地址决定帧在链路上的去向，ARP 报文内部的目标/发送方地址才承载“解析”语义。一开始看到请求里既有 ff:ff:ff:ff:ff:ff 又有全零，很容易以为二者是同一件事，把它们按层次拆开后，整个交换过程就顺理成章了。

第二点体会是长度要用字节去核对，而不是凭印象。ARP 报文 28 字节是把“8 字节固定字段 + 12 字节硬件地址 + 8 字节协议地址”算出来的，而帧长 42 与 60 的差别完全来自以太网最小帧长的填充要求。只有把这三段分开，才能解释为什么“同一个协议、同样的报文长度”会有两种帧长。

第三点是关于实验条件的如实记录。本次没有管理员权限，无法清空 ARP 缓存，工作负载也不是手册建议的浏览器访问，因此报告里只报告观测到的事实，不去推断“这次请求由哪次访问触发”。迹线本身的字段分析不受影响，但把偏差写清楚，比给一个看似完整的叙述更可靠。

最后是脚本复核的价值。报告中的帧数、报文长度、地址取值与十六进制字节，都是 analyze.py 直接读取 pcapng 原始字节重新解析得到的，与 Wireshark 界面显示互为独立验证，而不是从截图抄写。

七、附录

附录 A 迹线与复核文件

文件	说明
Lab4_ARP.pcapng	本次原始迹线，53 帧，全部为 ARP
evidence/0*.png	报告中使用的 7 张 Wireshark 截图与 3 张命令行截图
evidence/fig_arp_*.png	两张依据实测数据绘制的示意图
facts.json	脚本从 pcapng 原始字节解析出的全部关键数值
analyze.py	字段提取与长度核算脚本
make_diagrams.py	两张示意图的绘制脚本
ping_gateway.txt	工作负载 ping 的输出

表 4 附录文件清单

附录 B 关键命令与过滤器

```
# 取址与网关
ipconfig /all
```

```
Get-NetRoute -DestinationPrefix '0.0.0.0/0'

# 查看 ARP 缓存
arp -a

# 工作负载（绑定 WLAN 源地址访问网关）
ping -S 172.31.240.50 172.31.0.1

# Wireshark 过滤器
arp                                          # 捕获过滤器
eth.addr == 14:ac:60:56:df:85                  # 手册 Step 2 的显示过滤器
arp.isprobe || arp.isannouncement              # 探测与免费 ARP
arp.src.proto_ipv4 == 172.31.0.1 || arp.dst.proto_ipv4 == 172.31.0.1

# 命令行的等价核对
tshark -r Lab4_ARP.pcapng -T fields -e arp.opcode -e arp.src.hw_mac \
       -e arp.dst.hw_mac -e arp.src.proto_ipv4 -e arp.dst.proto_ipv4
```

附录 C　参考资料

[1] RFC 826: An Ethernet Address Resolution Protocol. https://www.rfc-editor.org/rfc/rfc826

[2] RFC 5227: IPv4 Address Conflict Detection. https://www.rfc-editor.org/rfc/rfc5227（ARP 探测与免费 ARP 通告）

[3] RFC 894: A Standard for the Transmission of IP Datagrams over Ethernet Networks. https://www.rfc-editor.org/rfc/rfc894（以太网最小帧长与填充）

[4] Wireshark Foundation: ARP display filter reference. https://www.wireshark.org/docs/dfref/a/arp.html